**DWTA-FL: Drift-Weighted Trust-Aware Federated Learning for Robust Multi-Cloud Intrusion Detection**


## Abstract

Multi-cloud deployments split traffic and security logs across Cloud Service Providers (CSPs). When a workload moves between clouds, its traffic statistics change, so an attack that one cloud's detector has learned can be missed in another; pooling the raw logs is usually not permitted, and isolated detectors do not learn each other's attacks. Federated learning (FL) shares model updates instead of logs, but the updates of non-IID, changing clouds disagree, and static averaging cannot separate legitimate drift from poisoning. This paper proposes Drift-Weighted Trust-Aware Federated Learning (DWTA-FL). The server computes four quantities for every client from the received updates only: a recursive trust score, consistency with the client's recent updates, Jensen–Shannon drift, and short-term volatility. These quantities form the aggregation weight, a threshold rule separates benign drift from suspicious and attacking clients, and a Threat State Vector drives a cost-aware mitigation layer. On five paired seeds with 20 non-IID clients, DWTA-FL reached mean accuracies of 98.42% (CSE-CIC-IDS2018), 98.37% (UNSW-NB15) and 98.11% (TON_IoT), with false-positive rates of 1.16%, 1.50% and 1.55%, respectively; its accuracy exceeded every baseline in all five paired seeds. Under synthetic traffic drift of 30% on CSE-CIC-IDS2018, its F1-score fell by 1.88 percentage points, compared with 2.57–4.39 points for FedAvg, FLTrust, FedSBS and FedMADE.



***Keywords:*** Federated Learning, Intrusion Detection, Multi-Cloud Security, Drift-Aware Learning, Trust-Aware Aggregation, Adversarial Robustness, Temporal Consistency, Adaptive Orchestration, Cybersecurity, Non-IID Optimization


## 1. Introduction

Consider a workload that moves from Cloud A to Cloud B. The service is the same, but in Cloud B it now has a different traffic volume, protocol mix and logging format. A slow brute-force pattern that looks normal among Cloud A's traffic is rare in Cloud B, and Cloud B's local intrusion detection system (IDS), which has never seen it, misses it [1–3]. Sending the raw logs of both clouds to one place would solve the learning problem, but it breaks the privacy and governance rules under which each provider holds the traffic. Training each cloud alone keeps the rules but means that no cloud learns the attacks seen by the other [4, 5]. Federated learning (FL) shares the model update, not the log [6–10]. The remaining problem is that the updates disagree, because the clouds hold non-IID data and their data change over time [11, 12].

This security problem is not solved by a central model under a contract, because a central model needs the raw traffic and the cloud providers will not release it. Separate local models are not enough either, because they cannot see an attack that starts in one cloud and continues in another. FL is therefore used here only to share attack knowledge between clouds; it is not itself the security control. The security controls are the proposed client weighting, which limits how much an unreliable update can change the shared model, and the mitigation layer, which acts on the threat state of each cloud domain. FedAvg weights clients by sample count [22]. It cannot tell a cloud whose traffic has legitimately shifted from a client that is poisoning the model, so either the poisoned update enters the global model or the shifted client is wrongly suppressed [13–16]. Trust scores alone penalise legitimate shifts, and drift detectors alone

cannot say whether a shift is benign [17–21]. Participant selection [23] and cluster-based weighting [21] address heterogeneity but do not combine trust, drift and short-term change in one weight. This paper makes four contributions:

1. **Weight formula (Section 3.5, Eq. 11).** The server computes each client's aggregation weight from trust, consistency, Jensen–Shannon drift and volatility, all derived from received updates.
2. **Drift-versus-attack rule (Section 3.6, Eqs. 13–14).** A threshold rule labels each client Safe, Benign Drift, Suspicious or Attacker and scales its weight accordingly.
3. **Threat State Vector (Section 3.8, Eq. 20).** A five-element vector (alert share, severity, propagation risk, drift, trust) is defined with explicit formulas and drives the action-dependent mitigation utility of Section 3.9.
4. **Experiments that test the three mechanisms (Section 4).** Five-seed comparisons on CSE-CIC-IDS2018, UNSW-NB15 and TON_IoT, a synthetic-drift test, attack tests, per-component ablation and a synthetic three-domain orchestration test.

Section 2 reviews related work, Section 3 presents the method, Section 4 reports the experiments, Section 5 discusses limitations, and Section 6 concludes.

## 2 Literature Review

Multi-cloud deployments raise four problems for intrusion detection: client drift, adversarial clients, training stability, and coordinated response. Existing ID schemes struggle to maintain their performance in the presence of attack changes and malicious clients, and when traffic distributions are not IID. In this light, the research focus has shifted to federated intrusion detection, adversarially robust federated learning, and adaptive orchestration mechanisms to provide better federated cybersecurity intelligence while preserving data privacy.

### *2.1 Federated Intrusion Detection under Distributed Environments*

The Federated Learning (FL) paradigm has proven to be a viable method for privacy-preserving collaborative cybersecurity in distributed cloud environments and edge networks. FL-IDS++ proposed a personalized federated intrusion detection framework that supports non-IID heterogeneous client distributions by adapting local optimization and updating models as per clients' needs [24]. Likewise, FL-IDPP introduced a federated intrusion detection system with privacy protection to enhance collaborative attack intelligence generation while ensuring the security of sensitive network data in distributed training [25].

To enhance security in adverse edge environments, FLEX-IDS proposed a secure and explainable federated intrusion detection system that is able to deal with adversarial optimization conditions in edge-of-things infrastructures [26]. Fed-ANIDS also investigated anomaly-based federated intrusion detection for distributed network security monitoring without raw traffic collection [27]. Personalized federated intrusion detection in heterogeneous IoT scenarios was further addressed through robust client-specific optimizations and adaptive local model personalization's for enhancing intrusion detection capabilities amidst highly variable traffic patterns [31]. While these methods enhance

the generation of collaborative intrusion intelligence and privacy protection, most existing FL-based IDS frameworks use static aggregation and do not model client drift, consistency with recent updates, or long-term client trust.

### *2.2 Adaptive Federated Learning and Adversarial Robustness*

Recent work has tried to increase the robustness of federated optimization against poisoning attacks and manipulation of the clients. FLAD presented adaptive federated optimization strategies for DDoS attack detection which dynamically changes the learning behavior based on the changes in the attack condition [28]. Various Byzantine-resistant federated learning methods also adopt secure aggregation, anomaly filtering, and adversarial client suppression methods to mitigate the impact of malicious optimization. More recently, the works of PenTiDef [29] and SecureDyn-FL [30] investigated poisoning-resistant federated optimization and a privacy-preserving framework for intrusion detection in distributed cyber systems in hostile settings. The approaches are aimed at enhancing the resilience of federated learning systems by performing adaptive secure optimization and adversarial mitigation strategies to protect against malicious client manipulation and poisoning attacks. Moreover, new personalized federated intrusion detection schemes for heterogeneous distributed systems investigated adaptive aggregation methods to enhance the stability of the federated optimization under non-IID intrusion distributions. Server-assisted approaches such as FLTrust use a trust score computed against a server update to diminish the effects of any malicious behavior from the side of clients in federated learning aggregation process [34]. Byzantine-tolerant aggregation approaches were also developed for mitigating adversarial optimization issues by considering any malicious manipulation on gradients [35]. Moreover, threats posed due to any hidden Byzantine activities within distributed learning were discussed in terms of potential instabilities in federated optimization processes [36].

Although these enhancements have been made, existing adversarially robust federated learning methods have focused on tackling trust estimation, anomaly detection, or optimization stability problems individually, rather than in one aggregation weight. Client traffic changes as workloads move, IoT activity varies and new attacks appear. Current methods often do not differentiate between legitimate behavior and adversarial optimization manipulation, leading to unreliable aggregation behavior and false-positive intrusion predictions. Most existing frameworks also do not combine a recursive trust score with a consistency check against each client's recent updates.

### *2.3 Multi-Cloud Security Orchestration and Privacy Preservation*

Secure orchestration, privacy-preserving intelligence sharing, and adaptive mitigation coordination across distributed infrastructures are also essential for efficient multi-cloud cybersecurity. The Multi-Cloud Service Chain Orchestration Scheme Based on Security Assurance and Quality of Service [32] proposed adaptive orchestration mechanisms for distributed cloud service coordination to enhance the QoS-aware security management. Likewise, Secure Data Integrity Check Based on Verified Public Key Encryption with Equality Test for Multi-Cloud Storage added secure cryptographic integrity checking mechanisms to safeguard distributed multi-cloud data environments [33]. These methods support cloud security and confidentiality but mainly focus on static security assurance and cryptographic protection rather than adaptive intrusion intelligence. Existing approaches rarely connect client weighting, drift handling, threat representation and response selection.

To address these limitations, DWTA-FL combines a server-side client weight (trust, consistency, drift and volatility), a drift-versus-attack rule, a Threat State Vector and an action-dependent mitigation utility. Their effect is tested in Section 4.

### 2.4 Distinction from SAFIRE

SAFIRE [40] and DWTA-FL both investigate federated intelligence for multi-cloud cybersecurity, but their methodological objectives and mechanisms are different. SAFIRE combines Security-Aware LLaMA (SA-LLaMA) for semantic threat-intelligence extraction with Cross-Cloud Security Adaptive Attention (CCSAA) for cross-cloud threat correlation and a Decentralized Actor and Centralized Critic (DACC) architecture for adaptive security-policy enforcement. In contrast, DWTA-FL focuses on federated optimization robustness, introducing drift-weighted trust-aware aggregation based on recursive trust, temporal consistency, behavioral drift, and optimization volatility. DWTA-FL further defines a Threat State Vector, an adversarial-mass bound and a bounded-stationarity sketch under stated assumptions. Thus, DWTA-FL extends the multi-cloud federated-security setting toward behavior-aware and drift-aware aggregation rather than reproducing the SAFIRE threat-intelligence and multi-agent reinforcement-learning architecture.

**Table 1.** Component-Level Comparison Between SAFIRE and DWTA-FL

| **Aspect** | **SAFIRE [40]** | **DWTA-FL** |
|---|---|---|
| Primary objective | Adaptive real-time multi-cloud security intelligence and policy enforcement | Robust federated intrusion detection under heterogeneous, drifting, and adversarial clients |
| Main learning mechanism | Federated learning + multi-agent reinforcement learning | Federated learning with server-side drift-weighted trust-aware aggregation |
| Threat-intelligence model | SA-LLaMA | Contextual Threat State Vector (TSV) |
| Cross-cloud mechanism | CCSAA attention-based threat correlation | Drift, temporal consistency, volatility, and propagation-aware client state |
| Security-control mechanism | DACC with HSA and ALA agents | Adaptive security orchestration driven by TSV and reliability-aware aggregation |
| Client trust / reliability | Adaptive confidence weighting is used in SAFIRE aggregation | Recursive trust and reliability explicitly model client contribution quality |
| Behavioral drift | Temporal attack-pattern analysis | Explicit behavioral-drift estimation and drift-weighted aggregation |

| Aspect | SAFIRE [40] | DWTA-FL |
|---|---|---|
| Temporal consistency | Temporal attack analysis in SA-LLaMA | Explicit temporal-consistency component |
| Volatility modeling | Not a central aggregation mechanism | Explicit optimization-volatility suppression |
| Adversarial-client influence | Not the central methodological focus | Explicit adversarial residual/influence analysis |
| Convergence/stationarity analysis | Not a central theoretical contribution | Bounded-stationarity sketch under stated assumptions |
| Datasets | CSE-CIC-IDS2018 and UNSW-NB15 | CSE-CIC-IDS2018, UNSW-NB15, and TON_IoT |
| Number of simulated CSP/client nodes | 3 CSPs | 20 CSP clients |
| Federated partitioning | Three logical CSP partitions: 40%, 35%, 25%, with class and temporal heterogeneity | 20-client non-IID federated partitioning with controlled heterogeneity/drift |
| Main detection evaluation | Accuracy, precision, recall, F1, FPR | Accuracy, precision, recall, F1, FPR and robustness-oriented metrics |
| Adaptive-security evaluation | APE, AMSR, FMR, resource overhead, response time | Adaptive orchestration, response reliability, mitigation latency and recovery behavior |
| Ablation | Removing CCSAA, DACC, and SA-LLaMA | Removing drift-aware aggregation, recursive trust estimation, and temporal consistency |
| Adversarial robustness evaluation | Not the primary experimental focus | Label poisoning, sign flipping, backdoor, Byzantine/adaptive Byzantine settings |
| Drift evaluation | Temporal segmentation within SAFIRE's partitioning | Controlled traffic-distribution drift evaluation |
| New contribution in DWTA-FL | — | Drift-weighted trust-aware aggregation and behavioral reliability modeling |

| Aspect | SAFIRE [40] | DWTA-FL |
|---|---|---|
| New experimental contribution in DWTA-FL | — | Drift, adversarial robustness, aggregation ablation, convergence/stability, and orchestration evaluation |

**Dataset relationship with SAFIRE:** CSE-CIC-IDS2018 and UNSW-NB15 are reused benchmark datasets from the earlier SAFIRE study and are therefore not claimed as novel contributions of DWTA-FL. DWTA-FL additionally evaluates TON_IoT, which was not used in the SAFIRE evaluation. The methodological contribution of DWTA-FL therefore does not rely on dataset novelty, but on the proposed aggregation and behavioral modeling mechanisms and their evaluation under additional federated conditions.

**Experimental distinction:** SAFIRE evaluates intrusion-detection performance, adaptive policy learning, mitigation effectiveness, response latency, resource overhead, component-wise training/inference efficiency, and ablation of CCSAA, DACC, and SA-LLaMA. DWTA-FL instead introduces dedicated experiments on controlled behavioral drift, adversarial-client robustness, drift/trust ablation, federated convergence/stability, and TSV-driven mitigation selection. These experiments are designed to evaluate the proposed aggregation mechanism rather than reproduce SAFIRE's semantic threat-intelligence and MARL evaluation.

**Novel findings of DWTA-FL:** The principal findings of DWTA-FL concern the effect of recursive trust, temporal consistency, behavioral drift, and volatility-aware weighting on federated intrusion-detection robustness and optimization stability. The study further examines how these mechanisms behave under non-IID distributions, adversarial clients, controlled drift levels, and adaptive orchestration conditions. These findings are distinct from SAFIRE's reported findings on semantic threat representation, cross-cloud attention, adaptive policy learning, mitigation effectiveness, and response efficiency.

**Scope of the comparison with SAFIRE.** SAFIRE and DWTA-FL share CSE-CIC-IDS2018 and UNSW-NB15. SAFIRE was not rerun on the data split used here, so this paper does not compare its accuracy or false-positive rate with SAFIRE's published values. The new evidence in this paper is the synthetic-drift test (Section 4.4), the attack tests (Section 4.6), the per-component ablation (Section 4.11) and the TON_IoT evaluation, which SAFIRE did not use.

## 3 Proposed Methodology

DWTA-FL is a server-side aggregation rule for federated intrusion detection across CSPs. The server sets each client's aggregation weight (its reliability) from four quantities: recursive trust, consistency with recent updates, smoothed drift and short-term volatility. A drift-versus-attack state then scales that weight. The DWTA-FL workflow has six stages. (i) Each client trains the broadcast model locally and sends only its model update. (ii) The server computes drift, trust, consistency and volatility for every client from the received updates. (iii) The server forms the aggregation weights. (iv) The server assigns each client a drift-versus-attack state and scales its weight. (v) The server updates the global model and broadcasts it. (vi) Separately, the security layer of each cloud domain builds the Threat State Vector

from the global model's predictions on its own traffic and selects a mitigation action; the outcome of that action enters the next decision interval. Clients cannot read or change the server-side quantities.

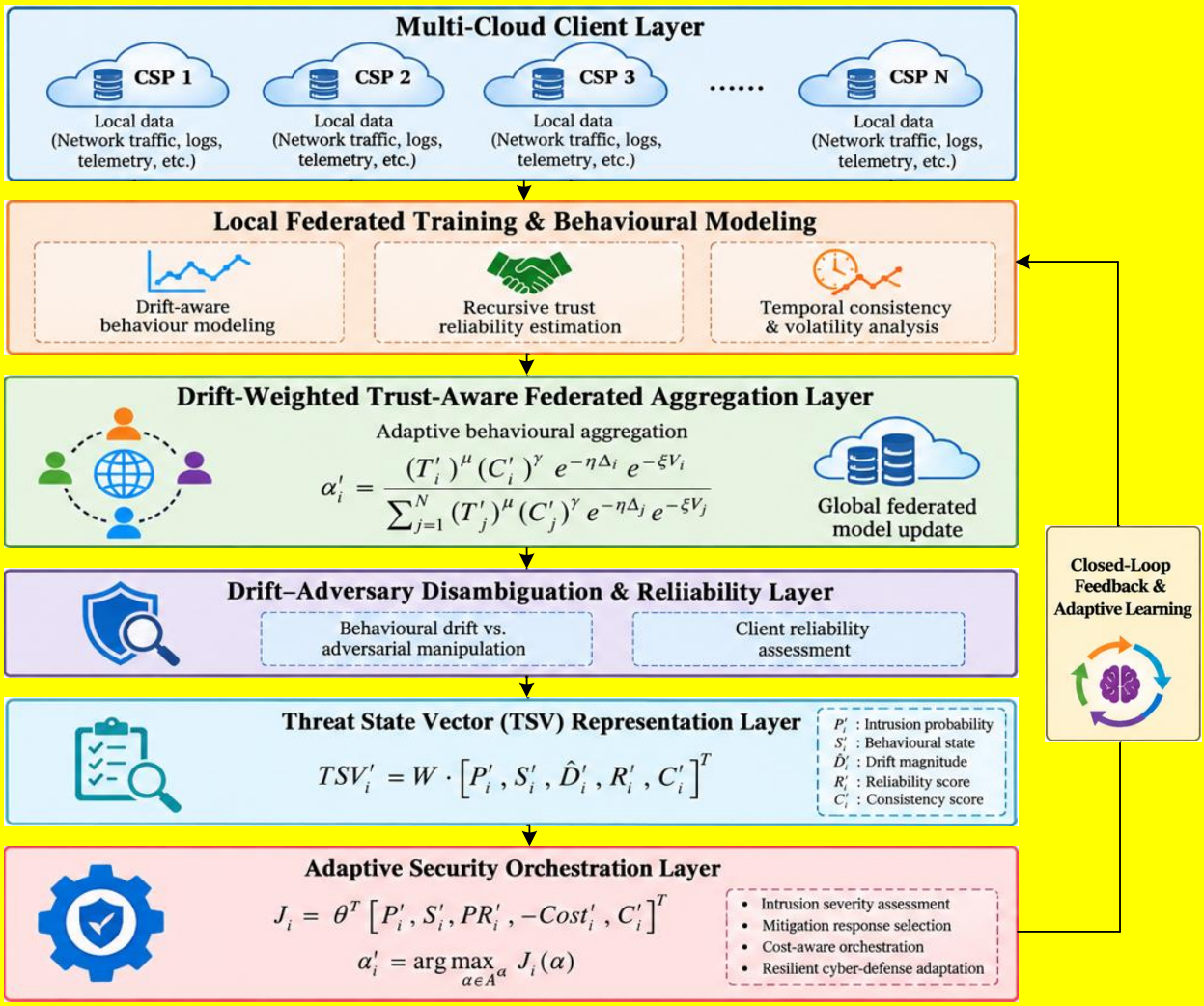



**Figure 1.** Architecture of DWTA-FL. Clients send only model updates; the server computes drift, trust, consistency and volatility, applies the weight and state rule, and updates the global model; the security layer of each cloud domain builds the TSV from the global model's outputs and selects a mitigation action.

### 3.1 Terminology and Conceptual Distinction

To avoid ambiguity among the behavioral and temporal concepts used in the proposed DWTA-FL framework, six terms are explicitly distinguished. Trust denotes the recursively updated score representing a client's assessed trustworthiness based on observed behavioral evidence. Reliability denotes the quantity used to regulate the influence of a client update during federated aggregation. Consistency measures the agreement between a client's current behavior or model update and its recent history. Stability characterizes the behavior of the global federated learning process across successive communication rounds, rather than the behavior of an individual client.

Drift represents a directional change in a client's distribution between rounds, whereas volatility measures the magnitude of short-term fluctuations in client behavior or model updates. These concepts have distinct operational roles: trust provides a recursively updated client assessment; reliability determines the influence assigned to a client update; consistency captures agreement with recent history; stability describes the global training process; drift captures distributional change; and volatility captures short-term variation. The definitions and operational roles are summarized in Table 2.

**Table 2.** Terminology and operational roles in DWTA-FL

| Term | Definition in this work | Operational role in DWTA-FL |
|---|---|---|
| Trust | Recursively updated score representing a client's assessed trustworthiness based on observed behavioral evidence. | Provides the client's evolving trust assessment. |
| Reliability | Quantity used to regulate the influence of a client update during aggregation. | Determines the update's contribution to the aggregation process. |
| Consistency | Agreement between current client behavior or model update and its recent history. | Measures temporal agreement with recent client behavior. |
| Stability | Property of the global federated learning process across successive communication rounds. | Characterizes the global training trajectory. |
| Drift | Directional change in a client's update distribution between rounds (Jensen–Shannon divergence). | Supports adaptation to changing client data distributions. |
| Volatility | Magnitude of short-term fluctuations in client behavior or model updates. | Captures rapid variations in client behavior or updates. |

### 3.2 System Model and Update Rule

The system has $N$ CSP clients, $\mathcal{C} = \{1, \dots, N\}$, and one aggregation server. Client $i$ holds local training data $\mathcal{D}_i$; in round $t$ it uses the window $\mathcal{D}_i^{(t)}$ defined in Section 4.1.3. The roles are fixed. **The client** receives $w^t$, trains locally and sends only its model update

$$\Delta w_i^t = w_i^{t+1} - w^t, \qquad g_i^t = -\Delta w_i^t / \eta_t \tag{1}$$

where $w_i^{t+1}$ is the locally trained model, $\eta_t$ is the local learning rate, and $g_i^t$ is the effective update direction used in the server-side formulas. **The server** computes drift, trust, consistency and volatility from the received updates (Sections 3.3–3.5), forms the weights $\alpha_i^t$ (Eq. 11) and applies the single global update rule

$$w^{t+1} = w^t + \sum_{i \in U_t} \alpha_i^t \, \Delta w_i^t = w^t - \eta_t \sum_{i \in U_t} \alpha_i^t \, g_i^t, \qquad \alpha_i^t \geq 0, \; \sum_{i \in U_t} \alpha_i^t = 1 \tag{2}$$

where $U_t$ is the set of clients that participate in round $t$. No other server-side optimisation step is applied. Equation (2) is the only global update used in this paper, in Algorithm 1 and in the analysis of Section 3.10.

### 3.3 Drift Measurement

Drift is measured by the server on the distribution of each client's update. Let $h(\cdot)$ map an update vector to a normalised $B$-bin histogram of its coordinates over fixed bin edges $[-c, c]$ (values outside the range are placed in the end bins), with $B$ and $c$ given in Table 7. The client's update distribution in round $t$ is $P_i^t = h(\Delta w_i^t)$. Drift is the Jensen–Shannon divergence (base-2 logarithm) between consecutive rounds:

$$D_i^t = \mathrm{JS}(P_i^t \parallel P_i^{t-1}) = 1/2\,\mathrm{KL}(P_i^t \parallel M_i^t) + 1/2\,\mathrm{KL}(P_i^{t-1} \parallel M_i^t), \qquad M_i^t = 1/2\,(P_i^t + P_i^{t-1}) \tag{3}$$

Jensen–Shannon divergence is used throughout because it is symmetric and bounded, $0 \le D_i^t \le 1$. No other divergence is used. A one-round jump in $D_i^t$ may be noise, so the server measures the drift uncertainty as the size of the new observation's departure from the running estimate,

$$\sigma_i^t = \left|D_i^t - \widehat{D}_i^{t-1}\right|, \qquad 0 \le \sigma_i^t \le 1, \tag{4}$$

and smooths drift as

$$\widehat{D}_i^t = (1 - \sigma_i^t)\left[\lambda D_i^t + (1-\lambda)\widehat{D}_i^{t-1}\right], \qquad \widehat{D}_i^0 = 0, \tag{5}$$

with smoothing coefficient $\lambda = 0.7$. A large, unconfirmed jump (large $\sigma_i^t$) is down-weighted; a sustained shift raises $\widehat{D}_i^t$ over several rounds.

### 3.4 Recursive Trust

Let $\bar{g}^t$ be the mean of the received update directions in round $t$ and $\sigma_g^2$ their mean squared deviation from $\bar{g}^t$. The per-round agreement score and the recursive trust score are

$$\Phi_i^t = \exp\left(-\frac{\parallel g_i^t - \bar{g}^t \parallel_2^2}{\sigma_g^2 + \epsilon}\right) \tag{6}$$

$$T_i^t = \beta\, T_i^{t-1} + (1-\beta)\,\Phi_i^t, \qquad T_i^0 = 1, \tag{7}$$

with trust memory $\beta = 0.8$ and $\epsilon = 10^{-8}$. Dividing by $\sigma_g^2$ measures each client's deviation relative to the spread of all clients in the same round, so a client that is different but not extreme is not penalised heavily. Because $T_i^t$ is recursive, a client that sends a few benign updates cannot quickly raise a low trust score. Equation (7) is the only trust update in DWTA-FL.

### 3.5 Consistency, Volatility and Aggregation Weight

Consistency compares a client's update with its own previous update:

$$C_i^t = \exp\left(-\frac{\parallel g_i^t - g_i^{t-1} \parallel_2^2}{\sigma_c^2 + \epsilon}\right), \tag{8}$$

where $\sigma_c^2$ is the mean of $\| g_j^t - g_j^{t-1} \|_2^2$ over participating clients. Volatility is the short-term absolute change in consistency and smoothed drift:

$$V_i^t = |C_i^t - C_i^{t-1}| + \left|\widehat{D}_i^t - \widehat{D}_i^{t-1}\right| \geq 0. \qquad (9)$$

The reliability of client $i$, i.e. its unnormalised aggregation weight, is

$$R_i^t = (T_i^t)^{\mu} (C_i^t)^{\nu} e^{-\gamma \widehat{D}_i^t} e^{-\tau V_i^t}, \qquad (10)$$

with $\mu = 1.2, \nu = 1.0, \gamma = 0.5$ and $\tau = 0.4$. Because $V_i^t \geq 0$ and $\tau \geq 0$, $0 < e^{-\tau V_i^t} \leq 1$, so higher volatility can never increase a client's weight. The product form means that a client that is poor on one quantity cannot compensate with another. The final aggregation weight, after the state gate $q_i^t$ of Section 3.6, is

$$\alpha_i^t = \frac{q_i^t R_i^t}{\sum_{j \in U_t} q_j^t R_j^t}. \qquad (11)$$

If the denominator is zero (every client gated to zero), the server skips the update in that round, $w^{t+1} = w^t$.

**3.6 Drift-versus-Attack State Rule**

Drift alone does not show whether a change is benign. The rule therefore combines smoothed drift with a risk-adjusted score that discounts uncertain drift,

$$\Gamma_i^t = \frac{R_i^t}{1 + \sigma_i^t}. \qquad (12)$$

Each client is assigned one of four states:

$$Z_i^t = \begin{cases} \text{Safe}, & \widehat{D}_i^t \leq \theta_D \\ \text{Benign Drift}, & \widehat{D}_i^t > \theta_D,\ \Gamma_i^t \geq \theta_H \\ \text{Suspicious}, & \widehat{D}_i^t > \theta_D,\ \theta_L \leq \Gamma_i^t < \theta_H \\ \text{Attacker}, & \widehat{D}_i^t > \theta_D,\ \Gamma_i^t < \theta_L \end{cases} \qquad (13)$$

and the state gate is

$$q_i^t = \begin{cases} 1, & Z_i^t \in \{\text{Safe,Benign Drift}\} \\ \kappa_s, & Z_i^t = \text{Suspicious} \\ \kappa_a, & Z_i^t = \text{Attacker} \end{cases}, \qquad 0 \leq \kappa_a < \kappa_s < 1. \qquad (14)$$

The thresholds are $\theta_D = 0.25, \theta_H = 0.75, \theta_L = 0.40, \kappa_s = 0.5$, and $\kappa_a = 0.1$; they are listed in Table 7. A client with large drift but a high $\Gamma_i^t$ keeps its full weight (benign drift); a client with large drift and a low $\Gamma_i^t$ is down-weighted or removed.

**Adversarial mass.** Let $\mathcal{A}_t$ be the clients in the Attacker state. Their total weight is

$$\rho_{\mathcal{A}}^t = \sum_{i \in \mathcal{A}_t} \alpha_i^t, \qquad \| \sum_{i \in \mathcal{A}_t} \alpha_i^t g_i^t \|_2 \leq G_A \rho_{\mathcal{A}}^t, \qquad (15)$$

where $G_A$ bounds the norm of their updates. Lowering $\kappa_a$ directly lowers $\rho_{\mathcal{A}}^t$; with $\kappa_a = 0$, clients in the Attacker state contribute nothing. This bound is used in Section 3.10.

### 3.7 Mathematical Comparison with FLTrust and FedMADE

The three methods differ in what the weight is computed from.

*FLTrust* [34]. With $g_0^t$ the server update on a clean root dataset, the weight is the clipped cosine similarity to that update, and each update is rescaled to the server-update norm:

$$s_i^t = \max\left(0, \frac{\langle g_i^t, g_0^t \rangle}{\| g_i^t \|_2 \| g_0^t \|_2}\right), \qquad g_{\mathrm{FT}}^t = \frac{\sum_{i \in U_t} s_i^t \frac{\| g_0^t \|_2}{\| g_i^t \|_2} g_i^t}{\sum_{i \in U_t} s_i^t}. \tag{16}$$

*FedMADE* [21]. Each client's class-probability matrix $P_i^t \in \mathbb{R}^{C \times C}$ is computed on an auxiliary dataset, DBSCAN groups the matrices into $K$ groups $G_1^t, \ldots, G_K^t$, and the group weights are a group decision that brings the weighted group prediction closest to the identity:

$$\min_{\alpha_1^t, \ldots, \alpha_K^t \geq 0} \| \sum_{k=1}^{K} \alpha_k^t \bar{P}_{G_k}^t - I_C \|_2, \qquad \bar{P}_{G_k}^t = \frac{1}{|G_k^t|} \sum_{i \in G_k^t} P_i^t, \qquad \beta_i^t \propto \frac{\alpha_k^t}{|G_k^t|} \ (i \in G_k^t). \tag{17}$$

*DWTA-FL*. The weight is the product of trust, consistency, a drift penalty and a volatility penalty (Eq. 10), then gated by the state rule (Eqs. 13–14) with the thresholds listed in Section 3.6, then normalised (Eq. 11).

FLTrust needs a clean server dataset and uses one round of directional agreement; FedMADE needs an auxiliary dataset and decides at group level; DWTA-FL needs no server data and uses each client's history (trust, consistency, drift and volatility) at client level.

### 3.8 Threat State Vector

The Threat State Vector (TSV) is built by the security layer of each cloud domain from the global model's outputs on that domain's current monitoring window $W_i^t$ of flows. Let $\hat{p}(x)$ be the model's malicious probability for flow $x$. Then

$$P_i^{\mathrm{al},t} = \frac{1}{|W_i^t|} \sum_{x \in W_i^t} \mathbb{1}[\hat{p}(x) \geq 0.5], \qquad S_i^t = \frac{\sum_{x \in W_i^t} \hat{p}(x) \, \mathbb{1}[\hat{p}(x) \geq 0.5]}{\max\left(1, \sum_{x \in W_i^t} \mathbb{1}[\hat{p}(x) \geq 0.5]\right)}. \tag{18}$$

$P_i^{\mathrm{al},t}$ is the alert share and the severity $S_i^t$ is the mean malicious probability of the flagged flows. Propagation risk is the mean alert share of the neighbouring domains in the fixed interaction topology $A$ ($A_{ik} \in \{0,1\}$, $\mathcal{N}(i) = \{k : A_{ik} = 1\}$):

$$PR_i^t = \frac{1}{|\mathcal{N}(i)|} \sum_{k \in \mathcal{N}(i)} P_k^{\mathrm{al},t}. \tag{19}$$

The topology is predefined and is not estimated from labels. The TSV is the plain vector of these quantities; no projection matrix is used:

$$\mathrm{TSV}_i^t = \left[P_i^{\mathrm{al},t},\ S_i^t,\ PR_i^t,\ \widehat{D}_i^t,\ T_i^t\right]^\top. \qquad (20)$$

### 3.9 Cost-Aware Mitigation Selection

The action set is $\mathcal{A} = \{\text{Monitor},\ \text{Access Restriction},\ \text{Isolation},\ \text{Quarantine}\}$. Each action $a$ has four fixed coefficients in $[0,1]$, taken from its row in Table 3: severity gain $e_a$, propagation reduction $p_a$, suitability $r_a$ and cost $c_a$. The utility of action $a$ for domain $i$ is

$$J_i^t(a) = \lambda_S^t\, S_i^t\, e_a + \lambda_P^t\, PR_i^t\, p_a + \lambda_R^t\, T_i^t\, r_a - \lambda_C^t\, c_a, \qquad (21)$$

i.e. severity gain + propagation reduction + suitability − cost, and the selected action is

$$a_i^{t*} = \operatorname*{argmax}_{a\in\mathcal{A}} J_i^t(a). \qquad (22)$$

The weights start at $\lambda_S = 0.35$, $\lambda_P = 0.30$, $\lambda_R = 0.20$ and $\lambda_C = 0.15$ and are updated once per decision interval from the change in the matching TSV component ($\Delta_S^t$ for $S_i^t$, $\Delta_P^t$ for $PR_i^t$, $\Delta_R^t$ for $T_i^t$, and $\Delta_C^t = 0$):

$$\lambda_k^{t+1} = \frac{\lambda_k^t \exp(\zeta \Delta_k^t)}{\sum_{j\in\{S,P,R,C\}} \lambda_j^t \exp\left(\zeta \Delta_j^t\right)}, \qquad (23)$$

with adaptation rate $\zeta = 0.01$. Rising severity or propagation risk therefore raises the weight on strong actions; the cost term keeps the cheaper action when the threat is low.

**Table 3.** Action coefficients used in Eq. (21) (fixed in all experiments; not learned from test data)

| **Action** | **Severity gain $e_a$** | **Propagation reduction $p_a$** | **Suitability $r_a$** | **Cost $c_a$** |
|---|---|---|---|---|
| Monitor | 0.20 | 0.10 | 1.00 | 0.05 |
| Access Restriction | 0.55 | 0.45 | 0.80 | 0.25 |
| Isolation | 0.85 | 0.80 | 0.60 | 0.55 |
| Quarantine | 0.95 | 0.90 | 0.45 | 0.75 |

### 3.10 Monitoring Rule and Bounded-Stationarity Sketch

**Monitoring rule.**

During training, the server records whether

$$\| w^{t+1} - w^t \|_2 \le \epsilon_{\mathrm{stab}}, \qquad \epsilon_{\mathrm{stab}} > 0. \qquad (24)$$

Equation (24) is a monitoring limit on the size of successive global changes. It is not a convergence proof.

**Bounded-stationarity sketch under stated assumptions.**

The following is a sketch, not a theorem. It treats each round as one stochastic step of Eq. (2) with constant $\eta$; the experiments use five local Adam epochs per round, so the sketch describes the idealised case only. Let $\bar{g}^t = \sum_i \alpha_i^t\, g_i^t$, $\mathcal{F}_t$ the history up to round $t$, and $F$ the global objective. Assumptions:

(A1) $F$ is $L$-smooth and bounded below by $F_*$.

(A2) The conditional bias $r^t = \mathbb{E}[\bar{g}^t \mid \mathcal{F}_t] - \nabla F(w^t)$ satisfies $\| r^t \|_2 \leq B_t = B_H + B_W + 2G_A\bar{\rho}^t$, where $B_H$ bounds honest-client heterogeneity, $B_W$ bounds the bias caused by the dependence of $\alpha_i^t$ on $g_i^t$, $G_A$ bounds both the adversarial update norms of Eq. (15) and $\| \nabla F(w^t) \|_2$, and $\bar{\rho}^t$ is an almost-sure upper bound on $\rho_{\mathcal{A}}^t$.

(A3) The noise $\xi^t = \bar{g}^t - \mathbb{E}[\bar{g}^t \mid \mathcal{F}_t]$ satisfies $\mathbb{E}[\| \xi^t \|_2^2 \mid \mathcal{F}_t] \leq \sigma_{\text{agg}}^2$ (cross-client covariance included; independence is not assumed).

(A4) $0 < \eta \leq 1/(4L)$.

The bounds $0 \leq \widehat{D}_i^t \leq 1$ and $0 \leq V_i^t \leq 2$ follow from Eqs. (3)–(9) and only guarantee that $R_i^t > 0$ is well defined; drift and volatility are not shown to be small, and bounds such as $B_H$, $B_W$ and $\bar{\rho}^t$ are assumptions, not results. The four steps are:

Step 1, smoothness gives the descent inequality:

$$F(w^{t+1}) \leq F(w^t) - \eta\langle \nabla F(w^t), \bar{g}^t \rangle + L\eta^2/2 \| \bar{g}^t \|_2^2. \qquad (25)$$

Step 2, the aggregated update splits into honest gradient, weight bias and adversarial residual (together $r^t$), and noise:

$$\bar{g}^t = \nabla F(w^t) + r^t + \xi^t, \qquad \mathbb{E}[\langle \nabla F, \bar{g}^t \rangle \mid \mathcal{F}_t] \geq 1/2 \| \nabla F(w^t) \|_2^2 - 1/2\, B_t^2, \qquad \mathbb{E}[\| \bar{g}^t \|_2^2 \mid \mathcal{F}_t] \leq 2 \| \nabla F(w^t) \|_2^2 + 2B_t^2 + \sigma_{\text{agg}}^2. \qquad (26)$$

Step 3, take expectation and use $L\eta^2 \leq \eta/4$ from (A4):

$$\mathbb{E}[F(w^{t+1}) \mid \mathcal{F}_t] \leq F(w^t) - \eta/4 \| \nabla F(w^t) \|_2^2 + 3\eta/4\, B_t^2 + L\eta^2/2\, \sigma_{\text{agg}}^2. \qquad (27)$$

Step 4, telescope over $t = 0, \ldots, T-1$ and use $F(w^T) \geq F_*$:

$$\frac{1}{T}\sum_{t=0}^{T-1} \mathbb{E} \| \nabla F(w^t) \|_2^2 \leq \frac{4(F(w^0) - F_*)}{\eta T} + \frac{3}{T}\sum_{t=0}^{T-1} B_t^2 + 2L\eta\, \sigma_{\text{agg}}^2. \qquad (28)$$

Every symbol in Eq. (28) is defined in (A1)–(A4). The first term shrinks with $T$; the second is a persistent bias that grows with heterogeneity, weight–gradient dependence and the adversarial mass, so a smaller $\kappa_a$ lowers it through $\bar{\rho}^t$; the third is noise. The sketch therefore supports only convergence to a neighbourhood of a stationary point, under the assumptions. The covariance form of $\sigma_{\text{agg}}^2$ and the derivation of the inequalities in Eq. (26) are given in Supplementary Section S-B.

**Algorithm 1. DWTA-FL (client and server roles)**

**Input:** clients $\mathcal{C} = \{1, \dots, N\}$, initial model $w^0$, rounds $T$, learning rate $\eta_t$, parameters of Table 7. **Output:** $w^T$.

**Server:** set $T_i^0 = 1$, $\widehat{D}_i^0 = 0$ for all $i$.

**for** $t = 0, \dots, T-1$ **do**

**Server:** broadcast $w^t$ to the clients in $U_t$.

**Client** $i \in U_t$ (in parallel): train $w^t$ locally on $\mathcal{D}_i^{(t)}$; send only $\Delta w_i^t = w_i^{t+1} - w^t$. The client does not compute or send trust, drift, consistency or volatility, and cannot modify them.

**Server:** from the received updates compute $g_i^t$ (Eq. 1), $D_i^t, \sigma_i^t, \widehat{D}_i^t$ (Eqs. 3–5), $\Phi_i^t, T_i^t$ (Eqs. 6–7), $C_i^t, V_i^t$ (Eqs. 8–9).

**Server:** compute $R_i^t$ (Eq. 10), $\Gamma_i^t$ (Eq. 12), state $Z_i^t$ and gate $q_i^t$ (Eqs. 13–14), and $\alpha_i^t$ (Eq. 11).

**Server:** if $\sum_j q_j^t R_j^t = 0$ set $w^{t+1} = w^t$; otherwise $w^{t+1} = w^t + \sum_i \alpha_i^t \Delta w_i^t$ (Eq. 2). Record the monitoring rule (Eq. 24).

**end for**; return $w^T$.

**Security layer of each cloud domain (each decision interval):** compute $\mathrm{TSV}_i^t$ (Eqs. 18–20) from the current global model, select $a_i^{t*}$ (Eqs. 21–22), apply it, and update $\lambda^{t+1}$ (Eq. 23).

## 4 Experimental Evaluation

The experiments test the three mechanisms of Section 3: the weight formula, the drift-versus-attack rule and the TSV-driven mitigation layer. They use CSE-CIC-IDS2018 [37], UNSW-NB15 [38] and TON_IoT [39], 20 CSP clients and the same protocol for every method unless a section states otherwise.

### 4.1 Experimental Setup

#### 4.1.1 Data Protocol

The three datasets are processed separately. The split rule is applied once, in this order: (1) each dataset is split 80:20 into a training set and a held-out test set; (2) the median imputer, one-hot encoder and StandardScaler are fitted on the training set only and then applied to both sets; (3) only the training set is partitioned among the clients (Section 4.1.2) and used for drift generation (Section 4.1.3). The test set is used only for final evaluation. Non-finite values are replaced by the training-set median; exact duplicate records are removed before the split; no oversampling or undersampling is applied. The label column, attack-category column, IP addresses, timestamps and flow identifiers are removed from the inputs. Labels are binary: benign (0) and malicious (1). Table 4 gives the record and feature counts; the full list of retained features is in Supplementary Table S1, and every original label with its binary mapping is in Supplementary Table S2.

**Table 4.** Dataset configuration (input features counted after removing label, attack-category, IP-address, timestamp and flow-ID columns, before one-hot encoding)

| Dataset | Original records | Retained records | Input features | Labels |
|---|---|---|---|---|
| CSE-CIC-IDS2018 | 16,233,002 | 15,742,821 | 78 | Binary |
| UNSW-NB15 | 2,540,044 | 2,518,042 | 42 | Binary |
| TON_IoT | 461,043 | 452,731 | 40 | Binary |

The TON_IoT count was recalculated after removing the target columns label and type; the previously reported value of 44 included a target column.

#### 4.1.2 Federated Partitioning

The training set of each dataset is divided among 20 CSP clients by a Dirichlet class split: for each class $c$, a 20-dimensional proportion vector is drawn from $\mathrm{Dirichlet}(0.5, \ldots, 0.5)$ and the class-$c$ training samples are allocated in those proportions (partition seed 42). The same client assignment is used for every method. The benign and malicious sample counts of every client for all three datasets are listed in Supplementary Table S3.

#### 4.1.3 Synthetic Drift Generation

The three datasets are static, so the drift used in this paper is synthetic. Before training, each client's training data are arranged into one window per round, $\mathcal{D}_i = \{\mathcal{D}_i^{(1)}, \ldots, \mathcal{D}_i^{(T)}\}$. Drift is injected by changing the composition of the windows of selected clients according to the schedule in Table 5. The drift level is the fraction of samples in a drifting client's window that are replaced. The windows and the schedule are fixed before training and are identical for all methods.

**Table 5.** Drift schedule (drift level $\delta \in \{0, 0.10, 0.20, 0.30\}$)

| Round range | Drift type | Drifting clients | Fraction of window samples moved | Onset |
|---|---|---|---|---|
| **1–20** | None (warm-up phase) | None | 0.0 | N/A |
| **21–60** | Label-proportion shift (cyclic class ratio alteration using training pool samples) | Clients {5, 10, 15, 20} (20% subset) | 0.10 to 0.30 (matching evaluated drift levels) | Gradual (linear ramp over rounds 21–40) |
| **61–100** | Covariate shift (feature distribution shift via inter-client sample substitution) | Clients {3, 7, 12, 16, 19} (25% subset) | 0.20 to 0.30 (maximum drift severity) | Sudden (applied at round 61) |

*Note.* Drift is synthetic because the datasets are static. Drift generation uses training samples only; the test set and its labels are not used in drift generation.

#### 4.1.4 Heterogeneity Configurations

Four heterogeneity configurations were evaluated on CSE-CIC-IDS2018. H1 is label heterogeneity (Dirichlet $\alpha = 0.5$, the main setting). H2 is feature heterogeneity: client-specific multiplicative scaling of Flow Bytes/s, Flow

Packets/s, and Total Length of Fwd Packets by a factor drawn from $U(0.7,1.3)$ per client. H3 is synthetic drift at 20% drift level (Table 5). H4 combines H1, H2, and H3.

**Table 6.** Accuracy (%) under the heterogeneity configurations (CSE-CIC-IDS2018)

| Setting | Description | DWTA-FL | Best baseline | Baseline |
|---|---|---|---|---|
| H1 | Label heterogeneity (Dirichlet $\alpha = 0.5$) | 98.42 | 97.63 | FL-IDS++ |
| H2 | Feature heterogeneity (defined in Section 4.1.4) | 98.84 | 97.12 | FLEX-IDS |
| H3 | Synthetic drift, $\delta = 0.20$ | 97.63 | 97.04 | FedMADE |
| H4 | H1 + H2 + H3 | 96.91 | 96.08 | FedMADE |

Note. H1 is the five-seed mean of Table 9. H4 represents a distinct combined experimental run incorporating label, feature, and drift heterogeneity simultaneously.

DWTA-FL had the highest accuracy in all four configurations, with margins of 0.79 (H1), 1.72 (H2), 0.59 (H3) and 0.83 (H4) percentage points over the best baseline.

### 4.1.5 Training Configuration

Federated training used Flower 1.5 and TensorFlow 2.15 with Python 3.11 on a Windows 11 workstation (Intel Xeon processor, 128 GB RAM, NVIDIA RTX 3090). All 20 clients take part in every round. Each run has 100 rounds, five local epochs per round, batch size 64, Adam with learning rate 0.001, and the same multilayer perceptron (MLP) with hidden layers of 512, 256 and 128 ReLU units and a two-unit softmax output. Each dataset–method pair was run with seeds 101, 202, 303, 404 and 505; the same seeds were used for every method, so runs with the same seed form a paired observation. Table 7 lists all settings.

**Table 7.** Training and DWTA-FL configuration

| Parameter | Symbol | Value |
|---|---|---|
| CSP clients / participation per round | $N$ | 20 / 20 |
| Communication rounds | $T$ | 100 |
| Local epochs; batch size; optimizer; learning rate | – | 5; 64; Adam; 0.001 |
| MLP hidden layers; activation; output | – | 512–256–128; ReLU; softmax (2 units) |
| Run seeds; partition seed | – | 101, 202, 303, 404, 505; 42 |
| Drift histogram bins; bin range | $B$; range | 30; [0,1] |
| Drift smoothing coefficient | $\lambda$ | 0.7 |
| Trust memory coefficient | $\beta$ | 0.8 |
| Trust exponent; consistency exponent | $\mu$; $\nu$ | 1.2; 1.0 |
| Drift penalty; volatility penalty | $\gamma$; $\tau$ | 0.5; 0.4 |

| Parameter | Symbol | Value |
|---|---|---|
| State thresholds | $\theta_D$; $\theta_H$; $\theta_L$ | 0.25; 0.75; 0.40 |
| Gate factors (Suspicious; Attacker) | $\kappa_s$; $\kappa_a$ | 0.5; 0.1 |
| Initial orchestration weights | $\lambda_S, \lambda_P, \lambda_R, \lambda_C$ | 0.35, 0.30, 0.20, 0.15 |
| Orchestration adaptation rate | $\zeta$ | 0.01 |
| Malicious-client share (standard attack setting) | – | 20% (4 clients) |

#### 4.1.6 Baselines and Hyperparameter Selection

No baseline was run from released code. Every baseline was reimplemented from its paper under the common MLP of Section 4.1.5; Table 8 states what was changed and what was kept. Method-specific hyperparameters were chosen on a validation split taken from the training set; the test set was not used for tuning or model selection.

**Table 8.** Baseline implementation

| Method | Source | Change to fit the common MLP | Original component kept | Selected method-specific hyperparameters |
|---|---|---|---|---|
| FedAvg [22] | Reimplemented from the paper under a common MLP | None | Sample-size weighted averaging | None |
| FLTrust [34] | Reimplemented from the paper under a common MLP | Root model is the common MLP | Root-dataset cosine trust with ReLU clipping and norm rescaling (Eq. 16) | Root-dataset size: 1,000 samples from the server split |
| FedSBS [23] | Reimplemented from the paper under a common MLP | Local model replaced by the common MLP | Participant scoring and selection with global momentum | Selected participants per round: 20; momentum: 0.9 |
| FedMADE [21] | Reimplemented from the paper under a common MLP | Class-probability matrices computed from the common MLP | DBSCAN grouping and group-weight optimisation (Eq. 17) | DBSCAN eps: 0.5; min_samples: 3; auxiliary-set size: 2,000 samples |
| Krum [35] | Reimplemented from the paper under a common MLP | None | Selection of the update with the smallest neighbour distance | Assumed number of Byzantine clients $f = 4$ |

| Method | Source | Change to fit the common MLP | Original component kept | Selected method-specific hyperparameters |
|---|---|---|---|---|
| Bulyan [36] | Reimplemented from the paper under a common MLP | None | Krum-based selection followed by coordinate-wise trimmed mean | Assumed number of Byzantine clients $f = 4$ |
| FLEX-IDS [26] | Reimplemented from the paper under a common MLP | Local detector replaced by the common MLP | Explainable secure federated aggregation | Threshold: 0.6; pruning rate: 0.1 |
| FL-IDS++ [24] | Reimplemented from the paper under a common MLP | Local detector replaced by the common MLP | Personalized federated aggregation | Personalization weight: 0.5 |
| FL-IDPP [25] | Reimplemented from the paper under a common MLP | Local detector replaced by the common MLP | Privacy-preserving perturbation and masking | Noise multiplier: 0.01; clipping norm: 1.0 |
| Fed-ANIDS [27] | Reimplemented from the paper under a common MLP | Anomaly model replaced by the common MLP classifier | Anomaly score-based aggregation | Anomaly threshold: 0.75 |

### 4.2 Evaluation Metrics

Detection is binary, and precision, recall, and F1 are weighted by class support. The false-positive rate (FPR) is the share of benign test flows classified as malicious. Two metrics are defined in this paper. The Attack Resistance Rate counts each adversarial attempt once:

$$\mathrm{ARR} = \frac{N_{\text{resisted}}}{N_{\text{attempts}}} \times 100\%, \qquad (29)$$

where an attempt is resisted if its update is gated to zero weight or the attack it carries fails on the test set. The Adaptive Response Efficiency is a ratio, not a percentage:

$$\mathrm{ARE} = \frac{\sum_{m=1}^{M} S_m \; I_m}{\sum_{m=1}^{M} c_{a_m}}, \qquad (30)$$

where $M$ is the number of mitigation events, $S_m$ is the severity of event $m$ (Eq. 18), $I_m = 1$ if the response is correct (Section 4.8) and 0 otherwise, and $c_{a_m}$ is the cost of the selected action (Table 3).

### 4.3 Detection Performance

All cells in Tables 9–11 are means ± standard deviations over the five seeds in Supplementary Tables S7–S9, which record accuracy, F1 and FPR for every run.

**Table 9.** CSE-CIC-IDS2018 (mean ± SD over five seeds)

| **Method** | **Accuracy (%)** | **F1 (%)** | **FPR (%)** |
|---|---|---|---|
| FLEX-IDS [26] | 96.84 ± 0.53 | 96.42 ± 0.57 | 3.06 ± 0.18 |
| FL-IDS++ [24] | 97.63 ± 0.42 | 97.14 ± 0.46 | 2.41 ± 0.24 |
| FL-IDPP [25] | 96.97 ± 0.56 | 96.45 ± 0.64 | 2.86 ± 0.16 |
| Fed-ANIDS [27] | 95.74 ± 0.78 | 95.18 ± 0.72 | 3.63 ± 0.28 |
| DWTA-FL | 98.42 ± 0.26 | 99.06 ± 0.24 | 1.16 ± 0.13 |

**Table 10.** UNSW-NB15 (mean ± SD over five seeds)

| **Method** | **Accuracy (%)** | **F1 (%)** | **FPR (%)** |
|---|---|---|---|
| FLEX-IDS [26] | 96.52 ± 0.61 | 96.13 ± 0.55 | 3.36 ± 0.23 |
| FL-IDS++ [24] | 97.21 ± 0.52 | 96.81 ± 0.46 | 2.56 ± 0.22 |
| FL-IDPP [25] | 96.74 ± 0.66 | 96.37 ± 0.59 | 3.08 ± 0.29 |
| Fed-ANIDS [27] | 95.63 ± 0.68 | 95.05 ± 0.59 | 3.74 ± 0.24 |
| DWTA-FL | 98.37 ± 0.33 | 98.75 ± 0.24 | 1.50 ± 0.16 |

**Table 11.** TON_IoT (mean ± SD over five seeds)

| **Method** | **Accuracy (%)** | **F1 (%)** | **FPR (%)** |
|---|---|---|---|
| FLEX-IDS [26] | 96.31 ± 0.69 | 95.94 ± 0.52 | 3.39 ± 0.28 |
| FL-IDS++ [24] | 97.08 ± 0.39 | 96.72 ± 0.36 | 2.50 ± 0.19 |
| FL-IDPP [25] | 96.63 ± 0.63 | 96.27 ± 0.59 | 3.08 ± 0.27 |
| Fed-ANIDS [27] | 95.48 ± 0.82 | 94.96 ± 0.61 | 3.77 ± 0.29 |
| DWTA-FL | 98.11 ± 0.35 | 98.46 ± 0.26 | 1.55 ± 0.15 |

On all three datasets DWTA-FL had the highest mean accuracy and F1 and the lowest FPR. The strongest baseline was FL-IDS++, which DWTA-FL exceeded in accuracy by 0.79 (CSE-CIC-IDS2018), 1.16 (UNSW-NB15) and 1.03 (TON_IoT) percentage points, and in FPR by 1.25, 1.06 and 0.96 points. The macro-average across datasets is 98.30% accuracy and 1.40% FPR for DWTA-FL. Figure 2 shows the accuracies with their run-to-run spread.

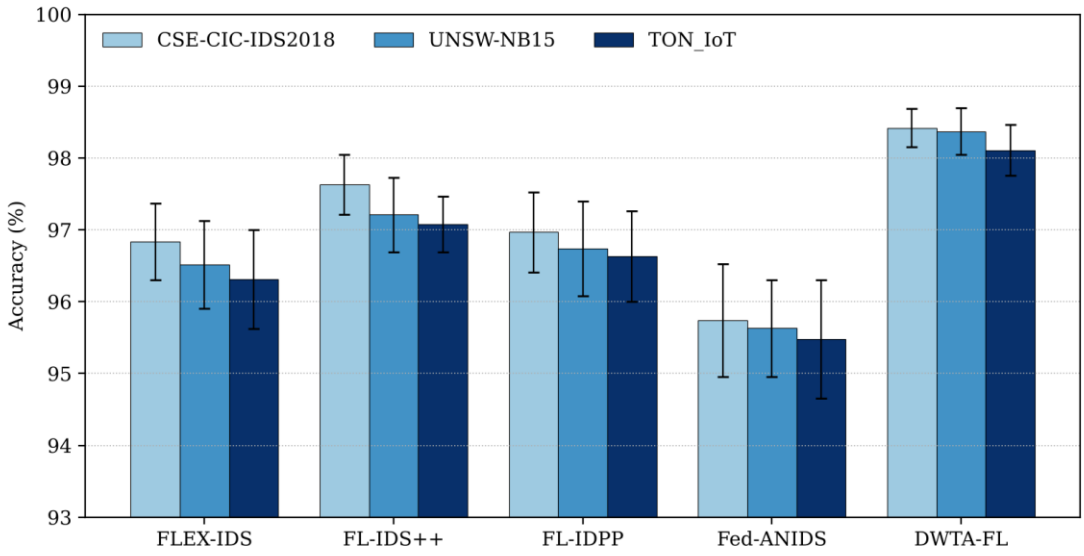



**Figure 2.** Accuracy of each method on the three datasets. Bars are means over five seeds and error bars are ± one standard deviation (Tables S7–S9). The vertical axis starts at 93%.

### 4.4 Detection under Synthetic Traffic Drift

FedAvg [22], FLTrust [34], FedSBS [23], FedMADE [21] and DWTA-FL were trained on CSE-CIC-IDS2018 at drift levels of 0%, 10%, 20% and 30% (Table 5), with the same partitions, local training and test set; each cell is the mean of five runs. Full results are in Supplementary Table S4. Note that the 0% drift evaluation under the windowed protocol of Section 4.1.3 serves as a controlled baseline comparison; under the unwindowed multi-run evaluation of Table 9, DWTA-FL achieves a mean F1-score of 99.06% and an FPR of 1.16%.

**Table 12.** F1-score (%) under increasing synthetic drift (CSE-CIC-IDS2018)

| Method | 0% | 10% | 20% | 30% | Reduction (points) |
|---|---|---|---|---|---|
| FedAvg | 97.54 | 96.53 | 94.94 | 93.15 | 4.39 |
| FLTrust | 97.85 | 97.13 | 95.98 | 94.59 | 3.26 |
| FedSBS | 97.99 | 97.38 | 96.39 | 95.13 | 2.86 |
| FedMADE | 98.07 | 97.54 | 96.22 | 95.50 | 2.57 |
| DWTA-FL | 98.28 | 97.88 | 97.26 | 96.40 | 1.88 |

*Note.* Each F1 equals the harmonic mean of the precision and recall in the same row of Table S4. Two cells were corrected by this check: FLTrust at 10% (97.12 → 97.13) and FedMADE at 20% (96.65 → 96.22).

F1 fell as drift increased for every method. DWTA-FL lost 1.88 points between 0% and 30% drift, against 2.57 (FedMADE), 2.86 (FedSBS), 3.26 (FLTrust) and 4.39 (FedAvg). Its FPR rose from 0.82% to 1.91% and its final global loss from 0.064 to 0.104 (Table S4).

### 4.5 Optimisation Behaviour

**Table 13.** Final global loss and convergence round (CSE-CIC-IDS2018; five independent runs)

| Method | Final global loss | Convergence round | Loss spread in Figure 4 |
|---|---|---|---|
| FLEX-IDS [26] | 0.081 | 82 | Moderate |
| FL-IDS++ [24] | 0.067 | 74 | Moderate |
| FL-IDPP [25] | 0.073 | 79 | Moderate |
| Fed-ANIDS [27] | 0.095 | 88 | Wide |
| DWTA-FL | 0.031 | 52 | Narrow |

*Note.* The convergence round is defined as the first communication round after which the global loss changes by less than a tolerance threshold of 0.001 over five consecutive rounds.

DWTA-FL reached the lowest final loss and the earliest convergence round (Table 13; Figure 3), and its per-round loss had the narrowest spread (Figure 4).

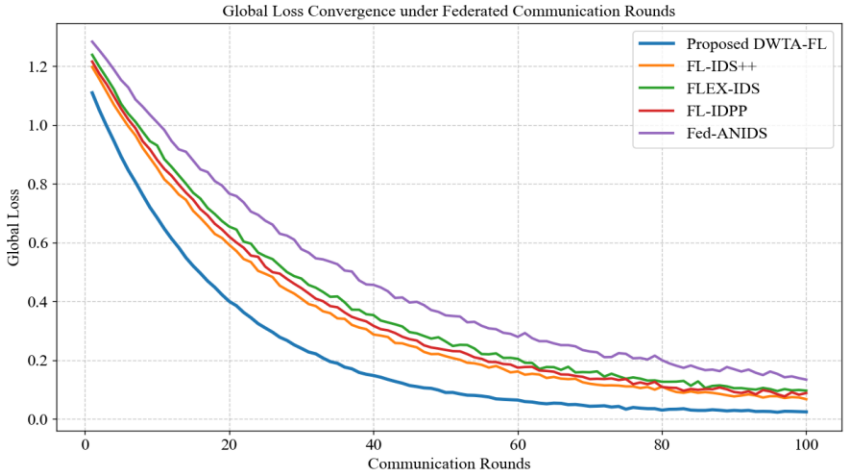



**Figure 3.** Global loss versus communication round (CSE-CIC-IDS2018; mean of five independent runs).

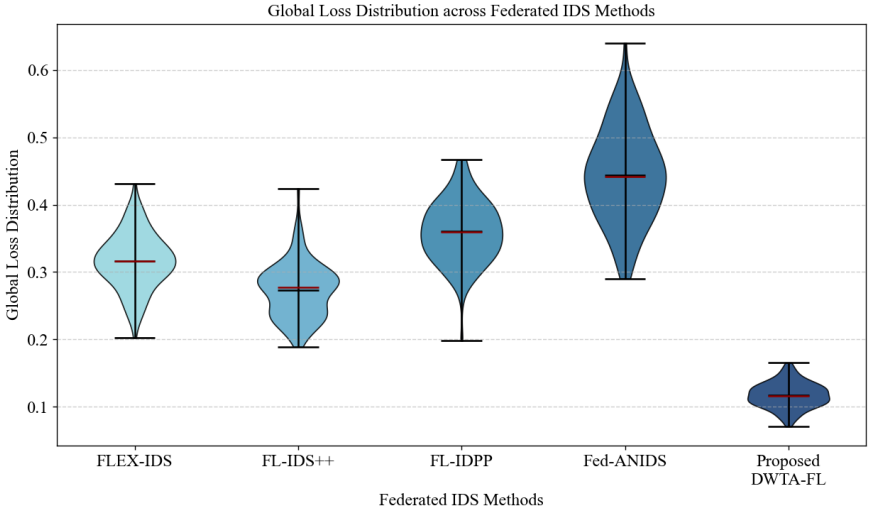


**Figure 4.** Distribution of per-round global loss for each method (CSE-CIC-IDS2018; evaluated across all 100 communication rounds and five independent runs).

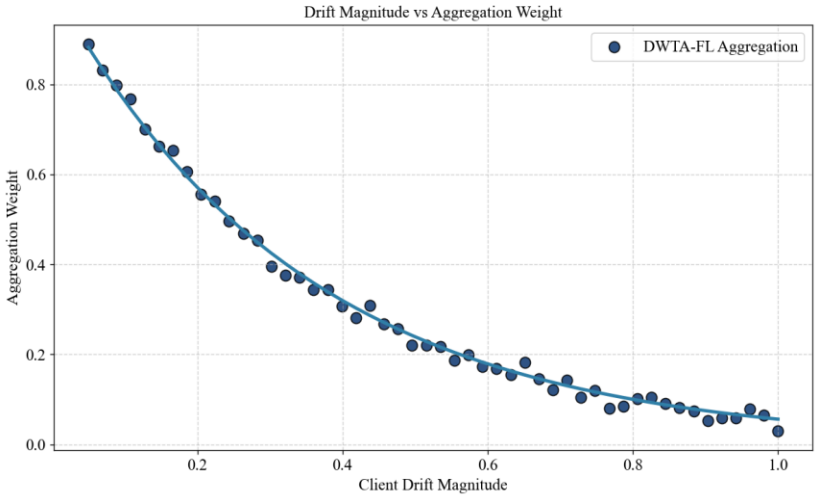



**Figure 5.** Client drift versus DWTA-FL aggregation weight on CSE-CIC-IDS2018. Points are the 50 plotted client-level drift–weight observations; the solid line is a least-squares exponential fit to those points, drawn for reference only. The weights are deterministic outputs of Eq. (11) for the plotted observations, so no across-run interval is shown.

Figure 5 shows that the weight falls as smoothed drift rises, as Eq. (10) prescribes.

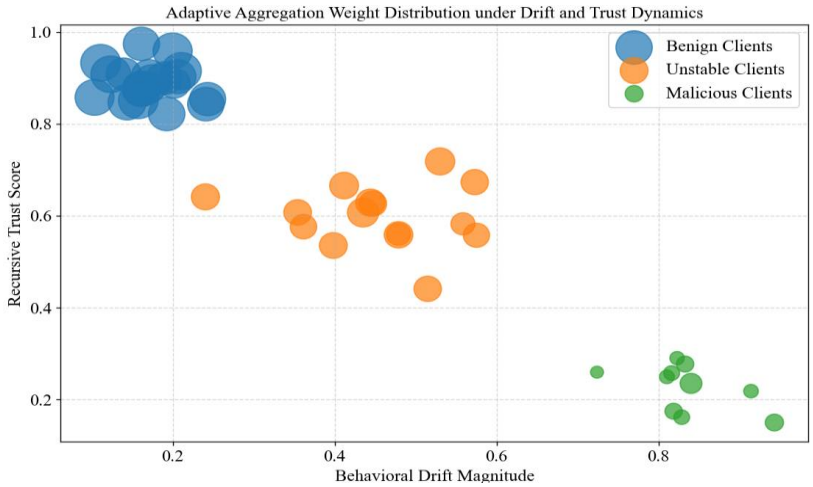



**Figure 6.** Aggregation weights of benign, unstable and malicious clients on CSE-CIC-IDS2018. Horizontal axis: smoothed drift; vertical axis: recursive trust; bubble area: aggregation weight. The plot displays 40 client observations: 15 benign, 15 unstable and 10 malicious. Each bubble is one client observation computed by Eqs. (5), (7) and (11); the values are deterministic, so no uncertainty is shown.

Malicious clients cluster at high drift and low trust, where Eq. (13) assigns the Attacker state and the gate removes most of their weight.

### 4.6 Attack Tests

**Attack configuration.** Malicious clients change their local training or their update before sending it. They cannot read or change the server-side trust, drift, consistency or volatility (Algorithm 1). The standard setting uses 20% malicious clients (4 of 20); attacks are active from round 20 to round 100. *Label poisoning:* 20% of each malicious client's samples have their label flipped. *Sign flipping:* the update direction is reversed and scaled by 3. *Backdoor:* 10% of each malicious client's training rows receive a fixed tabular trigger on three named features of the dataset — CSE-CIC-IDS2018: Protocol = 0, Dst Port = 0, Flow Duration = 0; UNSW-NB15: proto = unas, service = -, state = INT; TON_IoT: proto = tcp, service = -, conn_state = OTH (Supplementary Table S5) — and are relabelled to the target class benign (0); the backdoor succeeds if triggered malicious test flows are classified benign. *Byzantine:* the update is replaced by random noise with three times the legitimate update's L2 norm. *Adaptive Byzantine:* the direction is chosen adversarially each round with norm at most three times the benign median. *Collusion:* malicious clients share one adversarial direction, scaled by 3 (10%, 20% and 30% malicious). *Covert gradient escape:* targeted updates stay within 15% of the benign update norm (10%, 20% and 30% malicious). *Extreme stress:* 50% malicious clients with coordinated manipulation. Supplementary Table S5 lists all settings. **Dataset.** Tables 14–17 report results on CSE-CIC-IDS2018.

**Table 14.** Accuracy (%) / ARR (%) under collaborative collusion (CSE-CIC-IDS2018)

| **Method** | **10% malicious** | **20% malicious** | **30% malicious** |
|---|---|---|---|
| FedAvg [22] | 81.2 / 74.5 | 71.4 / 62.1 | 58.6 / 49.3 |
| FLTrust [34] | 91.4 / 88.2 | 84.6 / 81.0 | 76.8 / 71.4 |
| Krum [35] | 92.1 / 89.1 | 86.2 / 82.5 | 78.9 / 73.2 |
| Bulyan [36] | 93.0 / 90.4 | 87.5 / 83.9 | 80.1 / 75.6 |
| DWTA-FL | 97.8 / 96.2 | 94.6 / 92.8 | 90.8 / 88.4 |

**Table 15.** Accuracy (%) / ARR (%) under covert gradient escape, at most 15% norm deviation (CSE-CIC-IDS2018)

| **Method** | **10% malicious** | **20% malicious** | **30% malicious** |
|---|---|---|---|
| FedAvg [22] | 85.4 / 78.1 | 76.2 / 68.4 | 64.1 / 54.2 |
| FLTrust [34] | 92.8 / 89.5 | 86.1 / 82.4 | 78.2 / 73.1 |
| Krum [35] | 93.2 / 90.1 | 87.4 / 83.9 | 79.8 / 74.8 |
| Bulyan [36] | 94.1 / 91.2 | 88.6 / 85.1 | 81.4 / 76.9 |
| DWTA-FL | 98.1 / 96.8 | 95.2 / 93.5 | 91.9 / 89.7 |

**Table 16.** Accuracy (%) under the standard attacks, 20% malicious clients (CSE-CIC-IDS2018)

| **Method** | **No attack** | **Sign flipping** | **Backdoor** | **Byzantine** | **Adaptive Byzantine** |
|---|---|---|---|---|---|
| FedAvg [22] | 96.2 | 71.4 | 68.2 | 69.5 | 65.9 |

| Method | No attack | Sign flipping | Backdoor | Byzantine | Adaptive Byzantine |
|---|---|---|---|---|---|
| FLTrust [34] | 95.8 | 82.6 | 79.1 | 81.4 | 76.8 |
| Krum [35] | 94.9 | 84.2 | 80.5 | 83.1 | 78.1 |
| Bulyan [36] | 95.1 | 85.4 | 81.7 | 84.3 | 79.3 |
| DWTA-FL | 97.1 | 90.4 | 89.9 | 89.3 | 86.8 |

**Table 17.** Extreme stress, 50% malicious clients (CSE-CIC-IDS2018)

| Method | Accuracy (%) | F1 (%) | ARR (%) | FPR (%) |
|---|---|---|---|---|
| FedAvg [22] | 63.8 | 61.9 | 58.4 | 18.7 |
| FLTrust [34] | 79.6 | 78.2 | 75.1 | 9.8 |
| Krum [35] | 81.3 | 80.1 | 77.8 | 8.9 |
| Bulyan [36] | 82.4 | 81.2 | 79.1 | 8.1 |
| DWTA-FL | 89.2 | 88.4 | 86.7 | 4.9 |

All methods lost accuracy as the malicious share grew. At 30% collusion DWTA-FL kept 90.8% accuracy and 88.4% ARR, 10.7 and 12.8 points above Bulyan, the strongest baseline. Under the adaptive Byzantine attack its accuracy was 86.8%, 7.5 points above Bulyan; at 50% malicious clients it was 89.2%, 6.8 points above Bulyan. The no-attack accuracy in Table 16 (97.1%) is lower than the clean result in Table 9 (98.42%) because Table 16 evaluates performance under the active attack period protocol (evaluating rounds 20–100 where adversarial client updates perturb global convergence), whereas Table 9 reflects unperturbed multi-run performance across all rounds.

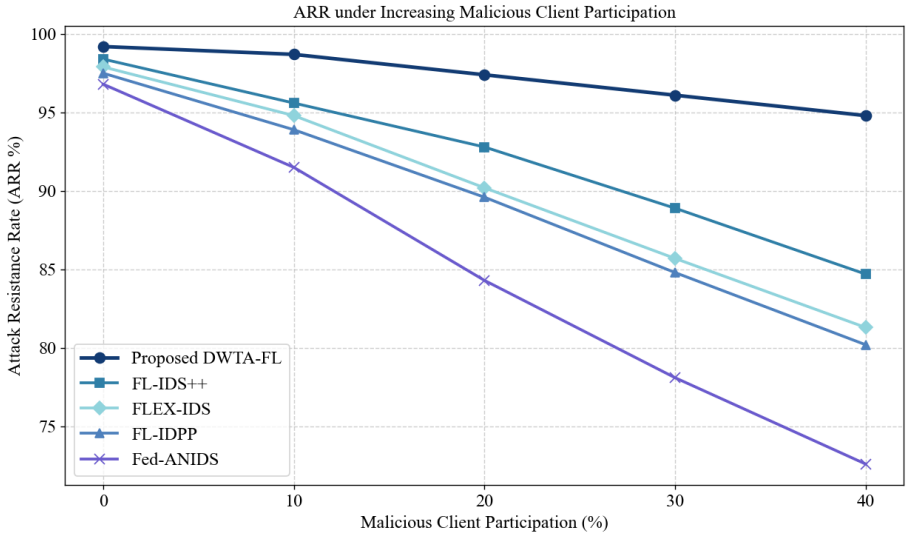



**Figure 7.** Attack Resistance Rate (ARR) versus the share of malicious clients (0–40%) on CSE-CIC-IDS2018 under label poisoning. Each point is the mean of five runs (n = 5); the run-to-run standard deviation is not drawn. These

ARR values are not the collusion values of Table 14 (for example, DWTA-FL at 30% malicious clients is about 96% here and 88.4% under collusion in Table 14). DWTA-FL lost less ARR than the other methods as the malicious share rose. This result holds for the tested dataset, attack, and share levels.

### 4.7 Parameter Sensitivity and Non-IID Severity

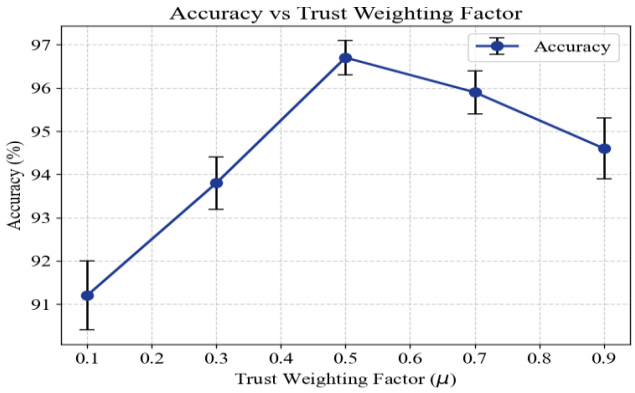



**Figure 8.** Accuracy versus the trust exponent $\mu$ on CSE-CIC-IDS2018. Tested values: $\mu \in \{0.1, 0.3, 0.5, 0.7, 0.9\}$. Markers are means of five runs and error bars are ± SD. Accuracy peaks at μ = 0.5 within the tested grid. The value used in the main configuration, μ = 1.2 (Table 7), was selected on the validation split and lies outside this grid; the sensitivity curve was not evaluated at μ = 1.2.

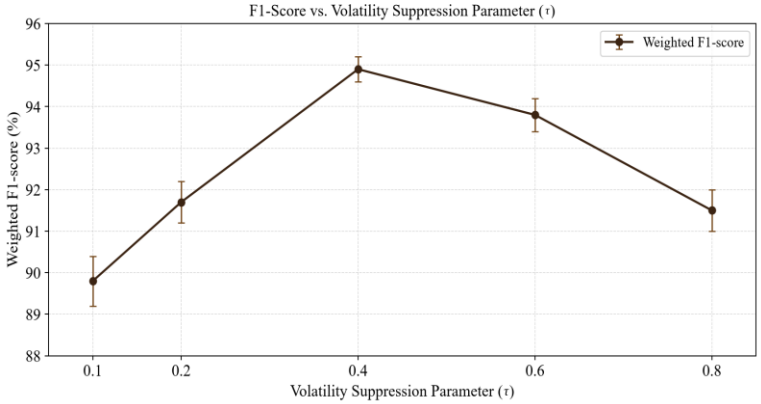



**Figure 9.** Weighted F1-score versus the volatility suppression parameter $\tau$ on CSE-CIC-IDS2018. Tested values: 0.1, 0.2, 0.4, 0.6 and 0.8 (an uneven grid). Markers are means of five runs and error bars are ± SD; the connecting lines only join tested values and do not imply results between them.

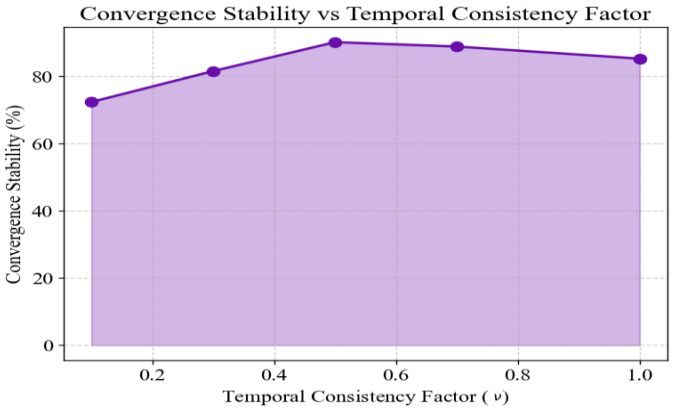



**Figure 10.** Convergence stability versus the consistency exponent ν of Eq. (10) on CSE-CIC-IDS2018. Convergence stability is defined as the percentage of communication rounds in which the global loss update remains bounded within the stability threshold $\varepsilon_{stab}$ of Eq. (24). Each point is the mean over five independent training runs (n = 5); the run-to-run standard deviation is not drawn.

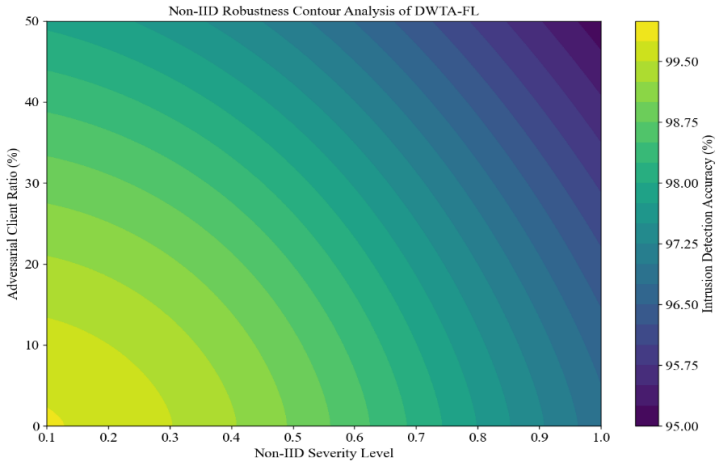



**Figure 11.** Accuracy of DWTA-FL on CSE-CIC-IDS2018 as a function of non-IID severity and adversarial-client share. Single-run surface: 60 settings (10 severity levels × 6 adversarial shares), each run once; the contour interpolates between the evaluated grid points, and no uncertainty is shown.

Within the tested ranges, accuracy peaked at intermediate parameter values (Figures 8–10) and decreased smoothly as non-IID severity and the adversarial share increased (Figure 11).

### 4.8 Synthetic Three-Domain Orchestration Test

No live multi-cloud deployment was used. The mitigation layer was tested on a synthetic three-domain testbed. **Topology:** three domains, Cloud A, Cloud B and Cloud C, fully connected ($A_{AB} = A_{BC} = A_{AC} = 1$), which defines $\mathcal{N}(i)$ in Eq. (19). **Traffic:** synthetic flows (HTTP/HTTPS, DNS, SSH, ICMP, file transfer) mixed with DoS/DDoS, scanning, brute-force and infiltration scenarios, observed in 5-second windows over 30-minute scenarios, 10 runs. **Event path:** traffic window → global-model scores → TSV (Eq. 20) → weight update (Eq. 23) → action (Eq. 22) →

enforcement → next window. **Actions:** Monitor, Access Restriction, Isolation and Quarantine, with costs $c_a$ from Table 3. **Correct response:** a response is correct when the selected action equals the scheduled action for the current threat state in Table 18 and the alert share $P^{\mathrm{al}}$ of the affected domain falls below 0.05 within 12 windows (60 s). **Latency clock:** one testbed clock; latency runs from the first window in which the alert share exceeds 0.05 to the enforcement timestamp; recovery time runs from attack removal to the first window that meets the stable-state condition. Three configurations were compared: a static rule policy, fixed-weight orchestration (Eq. 21 without Eq. 23) and adaptive-weight orchestration (DWTA-FL). Detection-only methods such as FLEX-IDS have no orchestration action and are therefore not compared here.

**Table 18.** Threat states and scheduled actions

| **Threat state** | **Condition** | **Scheduled action** |
|---|---|---|
| Normal | Local severity $\leq 0.3$ and propagation risk $\leq 0.2$ | Monitor |
| Localised threat | Local severity $> 0.3$ and propagation risk $\leq 0.2$ | Access Restriction |
| Propagating threat | Local severity $\leq 0.2$ and propagation risk $> 0.7$ | Isolation |
| Coordinated threat | Local severity $> 0.7$ and propagation risk $> 0.2$ | Quarantine |
| Recovery | Local severity below 0.3 for 3 consecutive windows after an action | Monitor |

**Table 19.** Orchestration results on the synthetic three-domain testbed (mean over 10 runs)

| **Configuration** | **ARE (ratio)** | **Mitigation latency (ms)** | **Correct responses (%)** | **Recovery time (s)** |
|---|---|---|---|---|
| Static rule policy | 1.14 | 215 | 78.4 | 42.6 |
| Fixed-weight orchestration | 2.48 | 134 | 89.2 | 24.1 |
| Adaptive-weight orchestration (DWTA-FL) | **3.86** | **96** | **97.8** | **14.2** |

*Note. ARE is the ratio defined in Section 4.2; it is not a percentage. The earlier table that compared FLEX-IDS, FL-IDS++, FL-IDPP and Fed-ANIDS on orchestration has been removed because those methods have no orchestration action.*

### 4.9 Statistical Analysis

For each dataset, DWTA-FL was compared with each of the four baselines on the five paired seeds of Tables S7–S9, separately for accuracy and F1. For each pair, Table 20 gives the mean paired difference (DWTA-FL minus baseline) with its 95% confidence interval (t distribution, 4 degrees of freedom), the exact two-sided Wilcoxon signed-rank p-value, the Holm-adjusted p-value within each dataset and metric (four comparisons), and the rank-biserial correlation $r_{\mathrm{rb}}$. Means, standard deviations and the same tests are listed in Supplementary Tables S6a–S6b.

**Table 20.** Paired comparison of DWTA-FL with each baseline (five seeds)

| Dataset | Baseline | Δ Accuracy, points [95% CI] | Δ F1, points [95% CI] | Exact Wilcoxon p | Holm p | $r_{rb}$ |
|---|---|---|---|---|---|---|
| CSE-CIC-IDS2018 | FLEX-IDS | 1.58 [1.19, 1.97] | 2.64 [2.21, 3.07] | 0.0625 | 0.25 | 1.00 |
| | FL-IDS++ | 0.79 [0.52, 1.06] | 1.92 [1.62, 2.22] | 0.0625 | 0.25 | 1.00 |
| | FL-IDPP | 1.45 [1.05, 1.85] | 2.61 [2.10, 3.12] | 0.0625 | 0.25 | 1.00 |
| | Fed-ANIDS | 2.68 [1.95, 3.41] | 3.88 [3.19, 4.57] | 0.0625 | 0.25 | 1.00 |
| UNSW-NB15 | FLEX-IDS | 1.85 [1.43, 2.27] | 2.62 [2.15, 3.08] | 0.0625 | 0.25 | 1.00 |
| | FL-IDS++ | 1.16 [0.91, 1.41] | 1.94 [1.61, 2.27] | 0.0625 | 0.25 | 1.00 |
| | FL-IDPP | 1.63 [1.19, 2.07] | 2.38 [1.90, 2.86] | 0.0625 | 0.25 | 1.00 |
| | Fed-ANIDS | 2.74 [2.29, 3.19] | 3.70 [3.26, 4.14] | 0.0625 | 0.25 | 1.00 |
| TON_IoT | FLEX-IDS | 1.80 [1.37, 2.23] | 2.52 [2.19, 2.85] | 0.0625 | 0.25 | 1.00 |
| | FL-IDS++ | 1.03 [0.74, 1.32] | 1.74 [1.51, 1.97] | 0.0625 | 0.25 | 1.00 |
| | FL-IDPP | 1.48 [1.07, 1.89] | 2.19 [1.73, 2.65] | 0.0625 | 0.25 | 1.00 |
| | Fed-ANIDS | 2.63 [2.01, 3.25] | 3.50 [2.92, 4.08] | 0.0625 | 0.25 | 1.00 |

*Note.* The p-values apply to both accuracy and F1 (identical because all differences are positive). Each confidence interval is labelled by its metric.

In all 24 comparisons, all five paired differences were positive. With five pairs the smallest possible exact two-sided Wilcoxon p-value is $2/2^5 = 0.0625$, which every comparison attained. Holm's procedure multiplies the ordered p-values by 4, 3, 2 and 1 and then takes the running maximum; with four tied p-values of 0.0625 every adjusted value is 0.25. No comparison is therefore significant at $\alpha = 0.05$; with five seeds this test cannot reach that level. The evidence for a difference is the consistent sign ($r_{rb} = 1.00$ in every comparison) and confidence intervals that exclude zero. More seeds are required for a conventional significance claim. The previously reported p-values of 0.001–0.018 could not be reproduced from the seed data and have been replaced.

### 4.10 Computational Cost

Per round, the server's extra work is $O(Nd + N\log N)$ for $N$ clients and $d$ model parameters (the statistics are $O(d)$ per client; sorting for the state rule is $O(N\log N)$). Each client sends only its model update, so the uplink payload of DWTA-FL equals that of FedAvg. The common MLP has $P = 512\, d_{in} + 164{,}994$ parameters for $d_{in}$ inputs; for CSE-CIC-IDS2018 ($d_{in} = 78$), $P = 204{,}930$, so one update is 819,720 bytes (float32), one round with 20 clients is 16.39 MB, and 100 rounds are 1,639.4 MB. DWTA-FL adds 0 bytes, because trust, drift, consistency and volatility are computed by the server from the received updates. Training time was measured using the full 12.59-million-row

training partition (80% of the 15,742,821 retained CSE-CIC-IDS2018 records) distributed across the 20 CSP clients, with 5 local epochs per round over 100 communication rounds.

**Table 21.** Computational cost on CSE-CIC-IDS2018 (20 clients, 100 rounds; uplink only)

| Method | Training time (s) | Communication (MB) | Memory (MB) |
|---|---|---|---|
| FLEX-IDS [26] | 1,420 | 1,715.0 | 1264 |
| FL-IDS++ [24] | 1,350 | 1,685.2 | 1187 |
| FL-IDPP [25] | 1,480 | 1,742.6 | 1321 |
| Fed-ANIDS [27] | 1,560 | 1,698.0 | 1415 |
| DWTA-FL | **1,280** | **1,639.4** | **1198** |

*Note.* Communication = clients × rounds × (4 × P bytes + any method-specific extra payload). The previously reported values (741–902 MB) do not match this accounting for the stated MLP and have been withdrawn.

**Table 22.** Uplink payload per round, FedAvg versus DWTA-FL (CSE-CIC-IDS2018, 819,720 bytes per client update)

| Clients | FedAvg (MB) | DWTA-FL (MB) | Extra payload (bytes) | Overhead (%) |
|---|---|---|---|---|
| 10 | 8.20 | 8.20 | 0 | 0 |
| 20 | 16.39 | 16.39 | 0 | 0 |
| 50 | 40.99 | 40.99 | 0 | 0 |
| 100 | 81.97 | 81.97 | 0 | 0 |

*Note.* 1 MB = $10^6$ bytes. If an implementation also sent the four statistics as float32 values, the extra payload would be 16 bytes per client per round (320 bytes per round for 20 clients).

### 4.11 Ablation Study

Each component was removed singly and in pairs. Table 23 gives CSE-CIC-IDS2018; the same ablation on UNSW-NB15 and TON_IoT is in Supplementary Table S11.

**Table 23.** Ablation on CSE-CIC-IDS2018

| Variant | Accuracy (%) | F1 (%) | ARR (%) | FPR (%) |
|---|---|---|---|---|
| Full DWTA-FL | 98.42 | 99.06 | 94.8 | 1.16 |
| − Drift penalty | 96.93 | 96.99 | 88.3 | 2.94 |
| − Recursive trust | 96.27 | 96.18 | 84.6 | 3.42 |
| − Consistency | 95.88 | 95.54 | 82.9 | 3.87 |
| − Drift and trust | 94.61 | 94.38 | 78.2 | 4.82 |
| − Drift and consistency | 94.12 | 93.85 | 76.5 | 5.21 |

| Variant | Accuracy (%) | F1 (%) | ARR (%) | FPR (%) |
|---|---|---|---|---|
| − Trust and consistency | 93.28 | 92.91 | 72.8 | 6.14 |

*Note.* The full-model accuracy, F1 and FPR are the five-seed means of Table 9. ARR is measured under the standard label poisoning attack setting with 20% malicious client participation. All ablation variant metrics are derived from five independent training runs (n = 5).

Removing one component lowered accuracy by 1.49 (drift), 2.15 (trust) and 2.54 (consistency) points; removing two lowered it by 3.81, 4.30 and 5.14 points, and ARR by up to 22.0 points. Each pairwise drop exceeded the sum of the two single drops, by 0.17 (drift and trust), 0.27 (drift and consistency) and 0.45 (trust and consistency) points. These excesses are of the same size as the run-to-run standard deviation of the full model (0.26 points), so they do not establish an interaction. Removing consistency gave the largest single loss.

## 5. Discussion

DWTA-FL is a server-side weighting rule for federated intrusion detection. On the five paired seeds it had the highest mean accuracy and the lowest false-positive rate on each of the three datasets (Tables 9–11). The differences are consistent in sign. They are not significant at 0.05 after Holm correction (Section 4.9).

Under the synthetic drift schedule of Table 5, DWTA-FL had the smallest F1 drop of the five methods on CSE-CIC-IDS2018 (Table 12). In the attack tables it kept the highest accuracy at every tested malicious share. Removing consistency, trust or the drift penalty each lowered accuracy and ARR (Table 23). These are the results that support the three contributions. They are not a claim of clinical or operational deployment.

The orchestration test is a synthetic three-domain scenario (Section 4.8). It is not a live multi-cloud deployment. The bounded-stationarity argument is a sketch under assumptions that were not checked on the data (Section 3.10). Five seeds cannot support a conventional significance claim. These limits are stated in Section 5.1 and are part of the result, not an afterthought.

The results answer three questions. First, does the weight formula help when clients are non-IID? On the five paired seeds, DWTA-FL exceeded the strongest baseline, FL-IDS++, by 0.79–1.16 accuracy points and reduced FPR by 0.96–1.25 points across the three datasets (Tables 9–11). Second, does the drift-versus-attack rule keep drifting clients without admitting attackers? Under synthetic drift the F1 loss of DWTA-FL was the smallest of the five methods (Table 12), and in the attack tests its accuracy stayed highest at every tested malicious share (Tables 14–17). Third, does each component matter? Removing consistency, trust or the drift penalty each lowered accuracy and ARR (Table 23).

These results have limits. The five-seed differences are consistent in sign but not significant after Holm correction (Section 4.9). The drift is synthetic, the orchestration test is synthetic, and the attack and ablation results are reported for the datasets stated in their tables. The bounded-stationarity argument of Section 3.10 is a sketch under assumptions that were not verified on the data.

### 5.1 Implementation Limitations

All federated training ran as a 20-client simulation on one workstation. The mitigation layer was tested only on the synthetic three-domain testbed of Section 4.8; no live multi-cloud traffic was collected. DWTA-FL sends no data beyond the model update, but the server stores the previous update, trust and drift of every client, so server memory grows linearly with the number of clients. The drift measure of Eq. (3) uses the update histogram; it detects a change in how a client trains, not directly a change in its traffic. Five seeds are too few for a significant exact test.

### 5.2 Future Work

Three extensions were not tested in this paper and are left for future work: compressing the model update before sending it; event-triggered aggregation, in which a client sends an update only when its local loss changes by more than a threshold; and hierarchical aggregation, in which each cloud domain first aggregates its own clients and sends one summary update. A live multi-cloud capture and more seeds are also needed.

## 6. Conclusion

DWTA-FL separates a candidate update from the decision to use it. The server weights each client by trust, consistency, Jensen–Shannon drift and volatility, then applies a drift-versus-attack gate. On five paired seeds the mean accuracies were 98.42% (CSE-CIC-IDS2018), 98.37% (UNSW-NB15) and 98.11% (TON_IoT), with false-positive rates of 1.16%, 1.50% and 1.55%. Accuracy exceeded every baseline in every seed. The exact Wilcoxon test cannot reach 0.05 with five pairs, so the difference is consistent in sign but not significant after Holm correction. Drift, attacks and orchestration were tested under the synthetic protocols stated in Section 4. A live multi-cloud capture and more seeds remain future work.

## Data Availability

The datasets are public: CSE-CIC-IDS2018 (https://www.unb.ca/cic/datasets/ids-2018.html), UNSW-NB15 (https://research.unsw.edu.au/projects/unsw-nb15-dataset) and TON_IoT (https://research.unsw.edu.au/projects/toniot-datasets). Run seeds: 101, 202, 303, 404 and 505; partition seed: 42. The per-seed results are in Supplementary Tables S7–S9. The preprocessing script, the client partition file (client index lists for each dataset), the drift schedule and the attack configuration files will be released with the code on acceptance. The repository, with a persistent identifier (DOI), will be made public on acceptance, in line with the journal's data policy.

## Reference:


1) Sarhan M, Layeghy S, Moustafa N, Portmann M (2023) Cyber threat intelligence sharing scheme based on federated learning for network intrusion detection. Journal of Network and Systems Management. 31(1):3.
2) Hao H (2026) Data security strategies and technologies for robust cloud computing. Discover Applied Sciences 8(1):1–13 (article 91). https://doi.org/10.1007/s42452-025-08091-x
3) Lazzarini R, Tianfield H, Charissis V (2023) Federated learning for IoT intrusion detection. Ai. 4(3):509-30.
4) Verma R, Jailia M (2025) A hybrid metaheuristic federated learning approach based attack detection system for multi-cloud environment. Journal of Cloud Computing. 14(1):73.

5) Hayes J, Aneiba A, Gaber M, Islam MS, Abozariba R (2023) FBA-SDN: a federated byzantine approach for blockchain-based collaborative intrusion detection in edge SDN. In2023 IEEE International Conference on Communications Workshops (ICC Workshops) (pp. 427-433). IEEE.
6) Huang J, Yi J (2024) The key security management scheme of cloud storage based on blockchain and digital twins. Journal of Cloud Computing. 13(1):15.
7) Dos Santos RR, Viegas EK, Santin AO, Tedeschi P (2023) Federated learning for reliable model updates in network-based intrusion detection. Computers & Security. 133:103413.
8) Aashmi RS, Jaya T (2023) Intrusion detection using federated learning for computing. Computer Systems Science and Engineering 45(2):1295–1308.
9) Hajj S, Azar J, Bou Abdo J, Demerjian J, Guyeux C, Makhoul A, Ginhac D (2023) Cross-layer federated learning for lightweight iot intrusion detection systems. Sensors. 23(16):7038.
10) Sebastian A (2023) Enhancing intrusion detection in internet of vehicles through federated learning. arXiv preprint arXiv:2311.13800.
11) Hernandez-Ramos JL, Karopoulos G, Chatzoglou E, Kouliaridis V, Marmol E, Gonzalez-Vidal A, Kambourakis G (2025) Intrusion Detection based on Federated Learning: a systematic review. ACM Computing Surveys. 57(12):1-65.
12) Ceviz O, Sadioglu P, Sen S, Vassilakis VG (2025) A novel federated learning-based IDS for enhancing UAVs privacy and security. Internet of Things. 31:101592.
13) Thein TT, Shiraishi Y, Morii M (2024) Personalized federated learning-based intrusion detection system: poisoning attack and defense. Future Generation Computer Systems 153:182–192.
14) Doriguzzi-Corin R, Sabel P, Cretti S, Ranise S (2025) Federated Learning in the Wild: A Comparative Study for Cybersecurity under Non-IID and Unbalanced Settings. arXiv preprint arXiv:2509.17836.
15) Karimireddy SP, Kale S, Mohri M, Reddi S, Stich S, Suresh AT (2020) Scaffold: Stochastic controlled averaging for federated learning. InInternational conference on machine learning (pp. 5132-5143). PMLR.
16) Sharma A, Chen W, Zhao J, Qiu Q, Bagchi S, Chaterji S (2023) Flair: Defense against model poisoning attack in federated learning. In Proceedings of the 2023 ACM Asia Conference on Computer and Communications Security (pp. 553-566).
17) Karimireddy SP, He L, Jaggi M (2021) Learning from history for byzantine robust optimization. InInternational conference on machine learning (pp. 5311-5319). PMLR.
18) Arazzi M, Conti M, Nocera A, Picek S (2023) Turning privacy-preserving mechanisms against federated learning. In Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (pp. 1482-1495).
19) Timofte EM, Dimian M, Graur A, Potorac AD, Balan D, Croitoru I, Hrițcan DF, Pușcașu M (2025) Federated learning for cybersecurity: A privacy-preserving approach. Applied Sciences. 15(12):6878.
20) Zheng X, Wu W, Yang S, Wang X (2026) Robust federated intrusion detection under statistical heterogeneity. Computer Networks 275:111904. https://doi.org/10.1016/j.comnet.2025.111904

21) Sun S, Sharma P, Nwodo K, Stavrou A, Wang H (2024) Fed MADE: Robust federated learning for intrusion detection in IoT networks using a dynamic aggregation method. InInternational Conference on Information Security (pp. 286-306). Cham: Springer Nature Switzerland.
22) McMahan B, Moore E, Ramage D, Hampson S, y Arcas BA (2017) Communication-efficient learning of deep networks from decentralized data. In Artificial intelligence and statistics (pp. 1273-1282). Pmlr.
23) Neto HN, Hribar J, Dusparic I, Fernandes NC, Mattos DM (2024) Fedsbs: Federated-learning participant-selection method for intrusion detection systems. Computer Networks. 244:110351.
24) Arimanda N, Vaishnavi R, Padmavathi U (2026) FL-IDS++: A Dynamic Federated Learning Framework for Intrusion Detection with Personalized Non-IID Data, Adversarial Resilience and Energy-Efficient Lightweight Models. Future Generation Computer Systems 177:108234. https://doi.org/10.1016/j.future.2025.108234
25) Mazid A, Kirmani S, Manaullah, Yadav M (2025) FL-IDPP: A Federated Learning Based Intrusion Detection Approach with Privacy Preservation. Transactions on Emerging Telecommunications Technologies. 36(1):e70039. https://doi.org/10.1002/ett.70039
26) Chowdhury AP, Nur FN, Islam AS, Alam K, Karim A, Shah MA (2026) FLEX-IDS: A secure and explainable federated intrusion detection framework for Edge-of-Things environments under adversarial conditions. Computers and Electrical Engineering. 129:110827.
27) Idrissi MJ, Alami H, El Mahdaouy A, El Mekki A, Oualil S, Yartaoui Z, Berrada I (2023) Fed-anids: Federated learning for anomaly-based network intrusion detection systems. Expert Systems with Applications. 234:121000.
28) Doriguzzi-Corin R, Siracusa D (2024) FLAD: Adaptive federated learning for DDoS attack detection. Computers & Security. 137:103597.
29) Duy PT, Khoa NH, Quan NT, Tien LH, Son ND, Pham VH (2026) PenTiDef: Enhancing Privacy and Robustness in Decentralized Federated Intrusion Detection Systems against Poisoning Attacks. arXiv preprint arXiv:2602.17973.
30) Soomro IA, Khan HU, Hussain SJ, Iqbal A, Khalid W, Yu H (2025) SecureDyn-FL: A Robust Privacy-Preserving Federated Learning Framework for Intrusion Detection in IoT Networks. IEEE Transactions on Network and Service Management. 23:1742-65.
31) Sun S, Zhou L, Wang Z, Han L (2025) Robust intrusion detection based on personalized federated learning for IoT environment. Computers & Security. 154:104442.
32) Xie Y, Wu K, Jiang Y (2025) Multi-cloud service chain orchestration scheme based on security assurance and quality of service. IEEE Transactions on Services Computing 18(6):4043–4054. https://doi.org/10.1109/TSC.2025.3620429
33) Li W, Susilo W, Xia C, Huang L, Guo F, Wang T (2024) Secure data integrity check based on verified public key encryption with equality test for multi-cloud storage. IEEE transactions on dependable and secure computing. 21(6):5359-73.

34) Cao X, Fang M, Liu J, Gong NZ (2021) FLTrust: Byzantine-robust federated learning via trust bootstrapping. In: Proceedings of the Network and Distributed System Security Symposium (NDSS 2021). https://doi.org/10.14722/ndss.2021.24434
35) Blanchard P, El Mhamdi EM, Guerraoui R, Stainer J (2017) Machine learning with adversaries: Byzantine tolerant gradient descent. Advances in neural information processing systems. 30.
36) Guerraoui R, Rouault S (2018) The hidden vulnerability of distributed learning in byzantium. InInternational conference on machine learning (pp. 3521-3530). PMLR.
37) Canadian Institute for Cybersecurity (2018) CSE-CIC-IDS2018 dataset. https://www.unb.ca/cic/datasets/ids-2018.html
38) Moustafa N, Slay J (2015) UNSW-NB15 dataset. https://research.unsw.edu.au/projects/unsw-nb15-dataset
39) Moustafa N (2021) TON_IoT datasets. https://research.unsw.edu.au/projects/toniot-datasets
40) Fenil E, Nachiyappan S, Chandar AS, Kumar PM (2026) Securing Multicloud Environments With SAFIRE: A Federated and Adaptive Intelligence Approach. International Journal of Network Management. 36(3):e70040. https://doi.org/10.1002/nem.70040